

ex of packages :- `java.lang` - it contains various classes used for language purpose & its default package so we don't need to use `import` package before using it.

2) `java.util` :- This package contains the utility classes & the data structures.

3) `java.sql` :- It contains the classes & interfaces which are used to connect java-backend with the database.

4) `java.io` - It contains the classes & interfaces for I/O used to perform i/p - o/p operation.

Friday
31/01/25

⇒ JVM :

- stands for Java virtual machine which are responsible for execution of byte code.
- JVM is the one which makes java platform independent with the help of byte code.
- we can execute a byte code on any platform which has JVM (JRE).
- JVM has several sub components such as
 - 1) class loader subsystem
 - 2) JVM memory areas
 - 3) execution engine.

① Class Loader Subsystem :-

Class loader subsystem of JVM is used to load a compiled class whenever we run a java program. It is first loaded by the class loader.

Architecture of JVM

classmate

Date _____
Page _____

Programmer

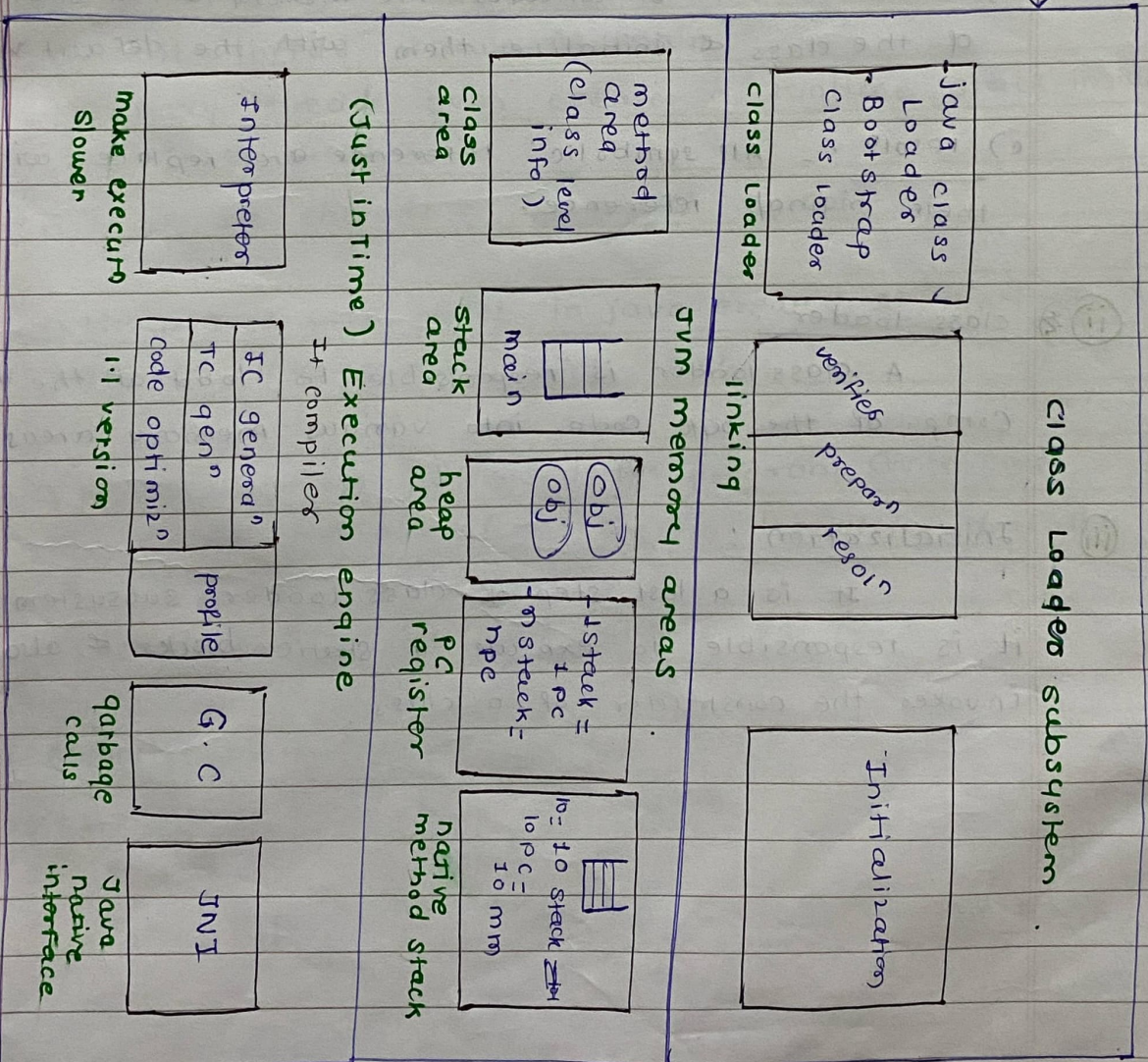
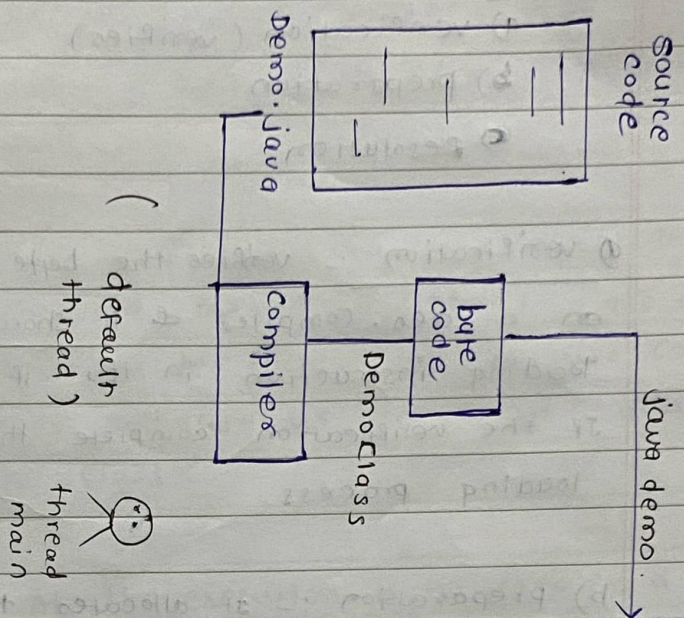


Fig:- Architecture of JVM

JVM Memory Areas :-

There are various memory areas in JVM such as class/method area, stack area, heap area, PC register & native method stack.

1) Class Area → This area stores all the class level info such as method, field, metadata about class etc.

- The members present inside this area are also called as shared resources because we are shared among all the instances of the class.

2) Stack area → At the execution JVM creates a default thread known as main.

For every thread JVM creates a runtime stack inside the stack area.

$n \cdot \text{no. of threads} = n \cdot \text{no. of stacks}$

3) Heap area → Object in Java created at runtime & stored into the memory area.

4) PC register → PC stands for Program Counter.

- It stores the address of the JVM instruction that the thread is currently executing.

- Each thread is allocated with 1 PC register.

5) Native method stack → This stack contains all the native methods which are required by the current executing thread.

- Each thread has a separate native method stack.

(i) Linking process :

- a) verification (verifies)
- b) preparation
- c) resolution

a) verification :- verifies the byte code is being compiled on a gen. compiler & it should contain the valid loading instruction in it, if not it throws "Linkage error". If the verification complete the class loader starts the loading process.

b) preparation :- it allocates the memory to all static var. of the class & initialize them with the default values.

c) resoln :- All symbolic reference are replace with their original reference.

(ii) class loader :

A class loader is responsible to load all the diff. Comp. of the byte code into various memory areas.

(iii) Initialization :

It is a last step of class loader subsystem where it is responsible to execute a static blocks & also invokes the constructor of a class.

Execution Engine :-

This component is mainly responsible for interpreting & execution of the byte code.

- It has several comp. init such as

- 1) Interpreter
- 2) JIT + profiles
- 3) GC
- 4) JNI

① Interpreter :

Interpreter is a slw which interprets the byte code line by line & check for logical mistake.

If a function is been called n no. of times the interpreter used to interprets the same function or times & make the execution slower & reduce the performance.

② JIT comp :

JIT stands for just in time compiler which was introduced in version 1.1 to improve the efficiency of interpreter.

- JIT is responsible to convert the byte code into machine understandable lang.

③ GC (Garbage Collector) :

This mechanism removes the unreference object from the heap area implicitly automatically.

- GC is responsible for dynamic memory allocation in java.

④ JNI :- Java Native Interface

- JNI is a framework which is used to communicate with the appn written in another lang. (native lang like c & c++)

Does java has destructor ?-

No, java doesn't have destruction better mechanism than destructor which is known as garbage collector which removes the unreferenced obj from heap area implicitly.